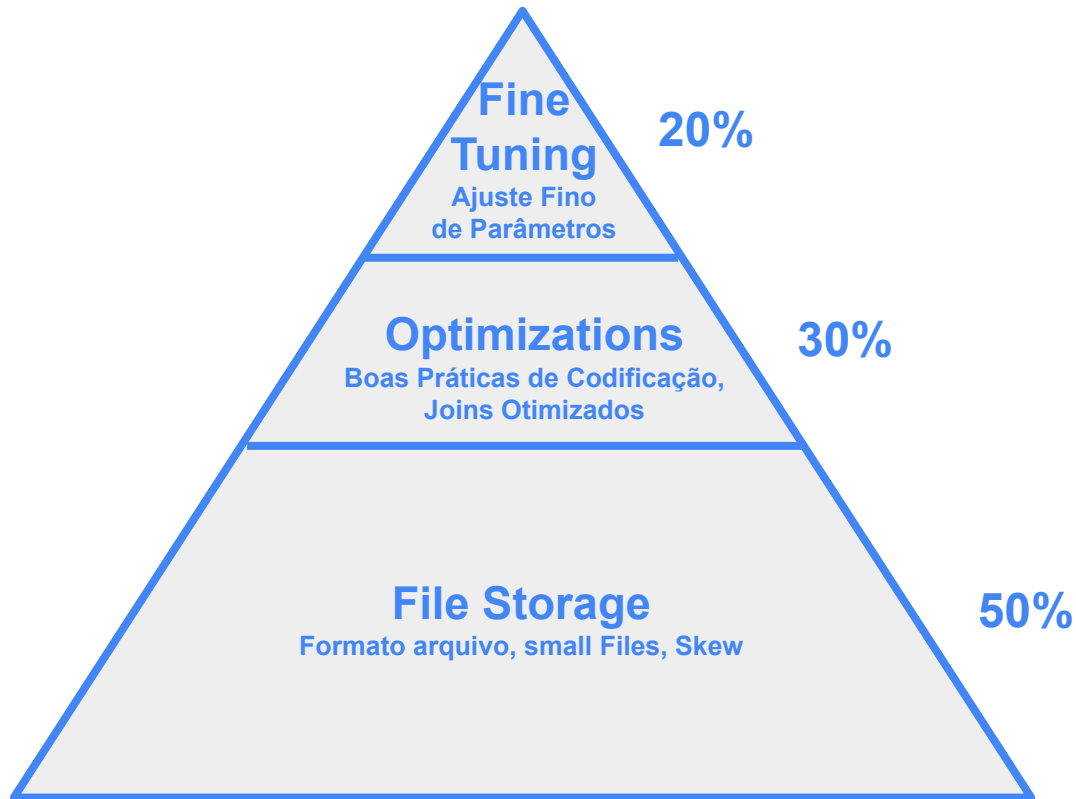


Spark Tuning

Os maiores problemas de Jobs Spark

Spark Tuning



Storage

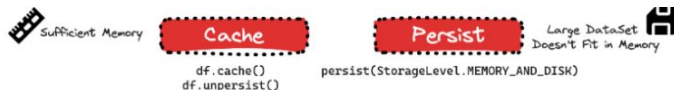
- Formato de Arquivo não otimizado para o propósito.
 - Formatos de recepção - Camada Raw

Format	Storage Type	Schema	Compression	Predicate Pushdown	Column Pruning	Ideal Use Case
JSON	Row-Based	No	External	No	No	Data Interchange, Flexible Schema
ORC	Columnar	Yes	Yes	Yes	Yes	Read-Heavy Analytical Workloads, Hive Integration
Parquet	Columnar	Yes	Yes	Yes	Yes	Most Analytical Workloads, Compatibility
AVRO	Row-Based	Yes	Yes	No	No	Data Serialization, Schema Evolution

- Iceberg (Formato Lakehouse)
 - Transações ACID
 - Time Travel e Histórico (Permite Rollback, Auditoria e Reprodução de Alterações)
 - Open Source (Comunidade enorme)
 - Permite Streaming e Batch
 - Full Schema Evolution Native (Não quebra quando você tem adição/remoção de colunas)
 - DML (Merge/Update e DELETE)
- Selecionar somente colunas que vai usar e usar filtros para reduzir a amostra (Column Pruning e Predicate Pushdown)
- Particionamento de Dados Eficiente (Foco em múltiplos de 32mb, 64mb, 128mb...)
- Usar o parâmetro `spark.sql.files.maxPartitionBytes` para Big Files e `repartition` logo no início para Small Files.

Shuffle

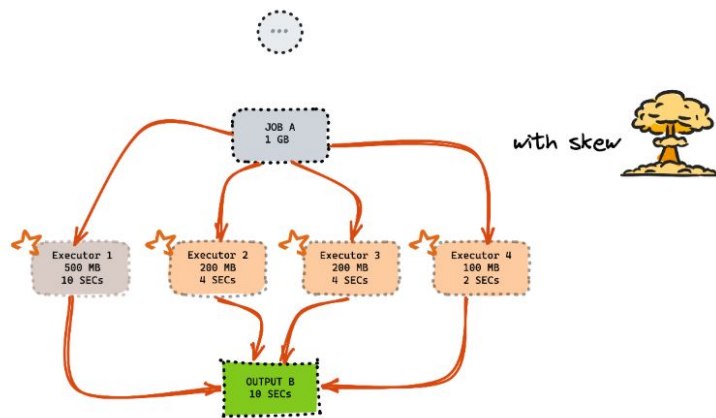
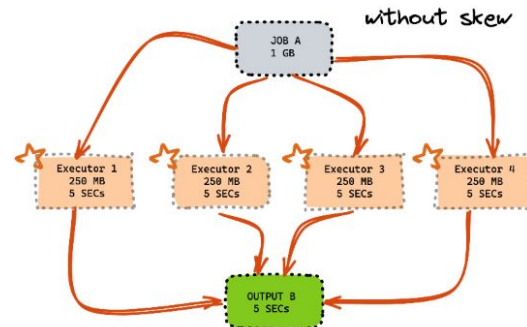
- Sempre ocorre quando temos uma **Wide Transformation** (Troca de dados entre os workers). Por exemplo, `groupBy`, `orderBy`, `repartition` e etc.
- Ações Úteis:
 - Usar Variáveis de Broadcast quando se tem um join entre um dataset grande e um dataset pequeno `df = broadcast(df)`.
 - Repartition/coalesce para quando se tem problemas na origem:
 - Regra Básica:
 - 3x a 4x a quantidade de cores total do cluster.
 - Por exemplo: Cluster com 8 cores = 32 partições.
 - Aumentar a memória e reduzir a quantidade de cores pode ajudar.
 - Reduzir a quantidade de operações wide desnecessárias (Em especial o `orderBy`)
 - Realizar cache de operações custosas com `persist()` e `cache`.



Storage Level	Execution	Description	Use Case
MEMORY_ONLY	<code>df.persist(StorageLevel.MEMORY_ONLY)</code>	Stores RDD as Deserialized Objects in JVM. Not Enough Memory, Partitions Will Not Be Cached & Will Need To Be Recomputed	Entire Dataset Fits In-Memory & Deserialization Cost is High
MEMORY_AND_DISK	<code>df.persist(StorageLevel.MEMORY_AND_DISK)</code>	Stores RDD as Deserialized Objects in JVM. Not Enough Memory, Partitions Will Be Stored On-Disk, Avoiding Recomputation	Dataset is Larger Than Memory But Eecomputation is Expensive
MEMORY_ONLY_SER	<code>df.persist(StorageLevel.MEMORY_ONLY_SER)</code>	Stores RDD as Serialized Java Objects, Space-Efficient Than MEMORY_ONLY, But Cost of Deserialization	Memory is Limited & Serialization Overhead is Acceptable
MEMORY_AND_DISK_SER	<code>df.persist(StorageLevel.MEMORY_AND_DISK_SER)</code>	Similar MEMORY_ONLY_SER But Spills Partitions That Don't Fit In-Memory to Disk	Memory is Limited & Dataset is Larger Than Memory, Can Tolerate Serialization Overhead
DISK_ONLY	<code>df.persist(StorageLevel.DISK_ONLY)</code>	Stores RDD Partitions Only on Disk	Memory is Very Limited or Accept Slower Disk Access
OFF_HEAP	<code>df.persist(StorageLevel.OFF_HEAP)</code>	Similar MEMORY_ONLY_SER But Stores RDD in Off-Heap Memory	When You Have Off-Heap Memory Available and Want Avoid JVM GC Overhead

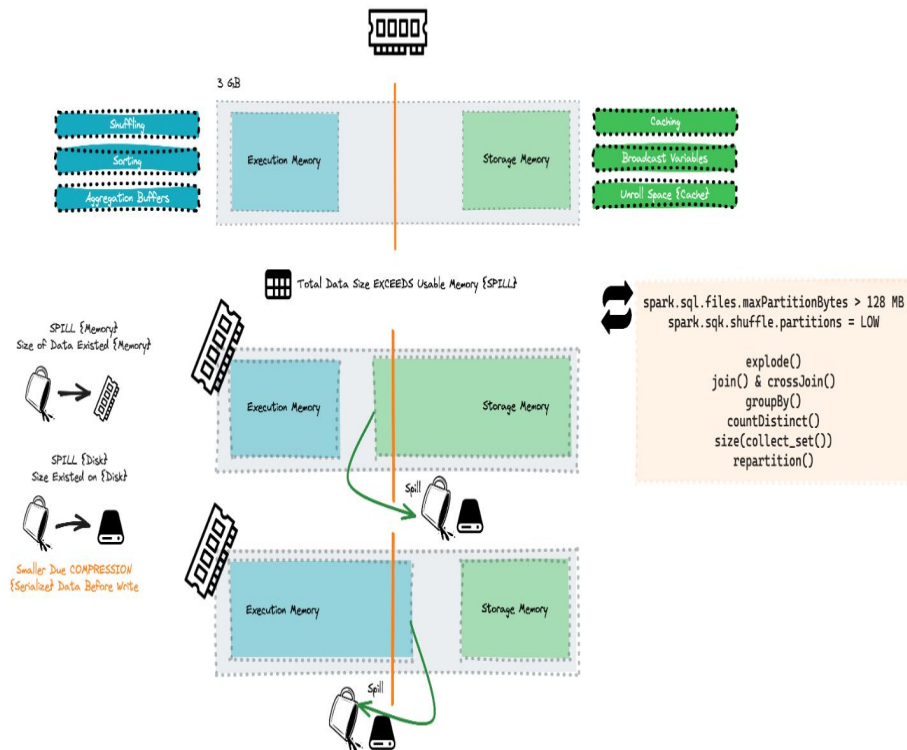
Skew

- Ocorre quando o dataset está desbalanceado na origem.
- Estratégias de Mitigação:
 - O ideal é resolver na origem (a depender do caso nao particionar)
 - Isolar o dado com skew e trabalhar ele separadamente.
 - Broadcast Hash Join
 - Usar Salting Keys ou `repartition` com uma custom key que nao seja desbalanceada.
 - Adaptive Query Execution Ligado



Spill e Serialization

- Ocorre quando uma transformação estoura a memória do spark e ele precisa escrever esses dados temporariamente em disco. Para fazer isso ele realiza uma operação de serialização e deserialização para compactar e descompactar esses dados.
- Estratégias de Mitigação
 - Aumentar a quantidade de workers (pode gerar shuffle).
 - Resolver problemas de Skew.
 - Gerenciar o partition size
`spark.sql.files.maxPartitionBytes`.
 - Evitar operações como explode(), crossjoin().
 - Evitar Python UDFs, de preferência para uma pandas UDF (que usa conceitos do pyarrow) ou funções nativas do spark (sempre o ideal).



Melhores Práticas - A “colinha”

- Extração:
 - Evitar formatos não otimizados para Big Data como JSON e TXT.
 - Resolver problemas de Data Skew na origem (partições desbalanceadas) otimizando o particionamento.
 - Prune Columns e Predicate Pushdown.
 - Broadcast Joins para datasets muito pequenos.
- Transformação:
 - Usar `cache` em dataframes intermediários (Lembrar de fazer `unpersist()`!).
 - Evitar wide transformations usando lambda e RDD.
 - Otimizar joins com `broadcast()` nos dataframes alvo.
- Escrita:
 - Otimizar a escrita usando Repartition ou Coalesce.
 - Particionamento (Se estiver no databricks, apenas acima de 1B linhas e usando liquid clustering, para os demais casos, só particionar quando o uso do `repartition/coalesce` não for suficiente para otimizar a leitura)
 - Usar formatos eficientes (Delta e Iceberg)